

Computer Architecture: *Memory Safety & Protection*

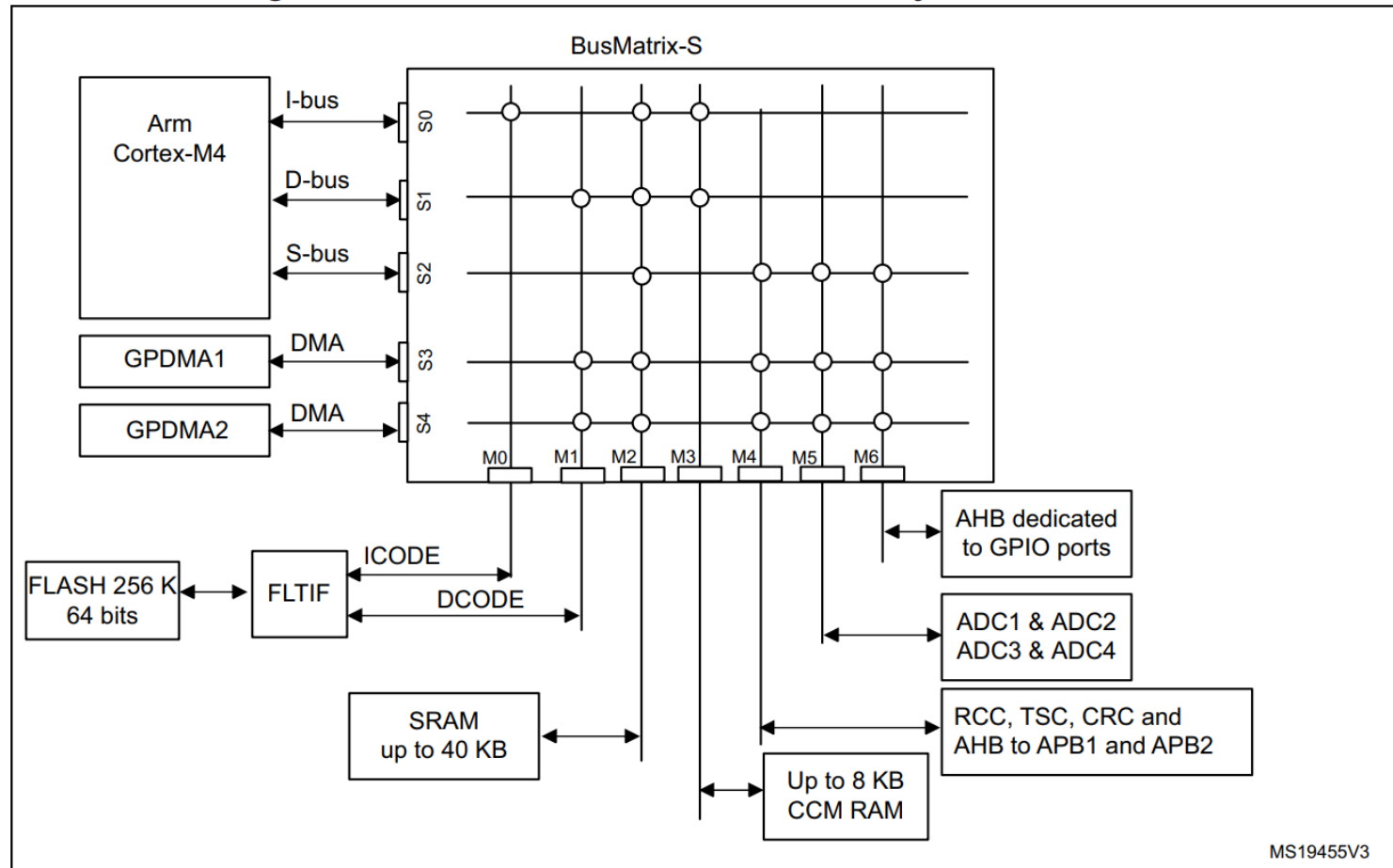
Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store data?
- How does the machine interface using buses?
- **What does it mean to have “allowed” memory?**
- How does the machine speed up data access?

Busses that don't respond or Invalid

Figure 1. STM32F303xB/C and STM32F358xC system architecture

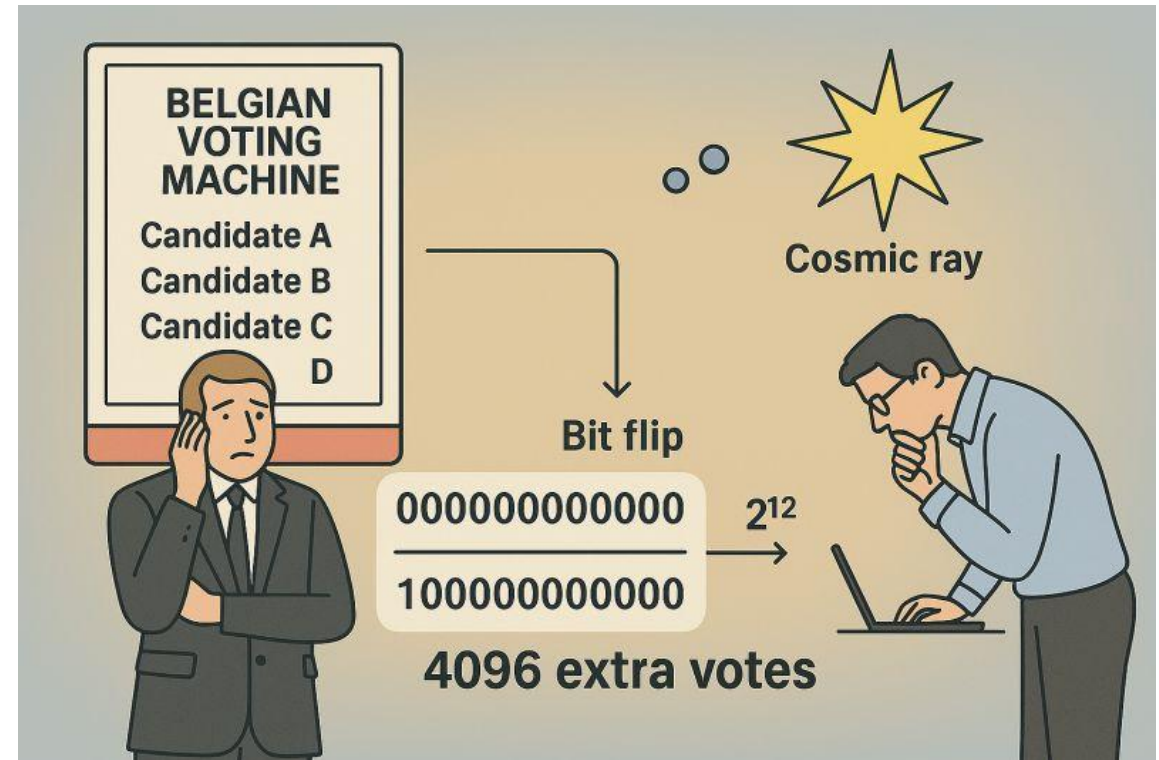
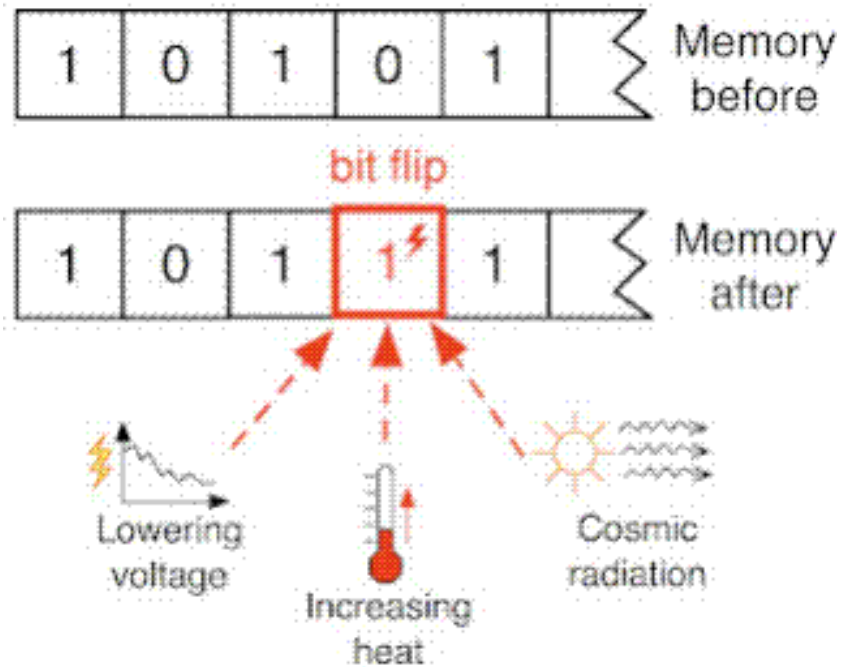


- Prior lecture looked at busses
- This lecture will go beyond just “non-existent”

Memory Safety

- Memory safety helps:
 - Protect against unintended natural events (bit-flips)
 - Protect against bugs that could cause dangerous operations
 - Protect against attackers exploiting the memory features

Memory Error Correction (ECC)



In 2003, a candidate in a Belgium election received 4096 extra votes... a suspicious number...

ECC Memory

To-be stored value:

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Hamming encoded value:

0	1	0	1	0	1	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

Recoverable Scenario with a hard-fault:

0	0	0	1	0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

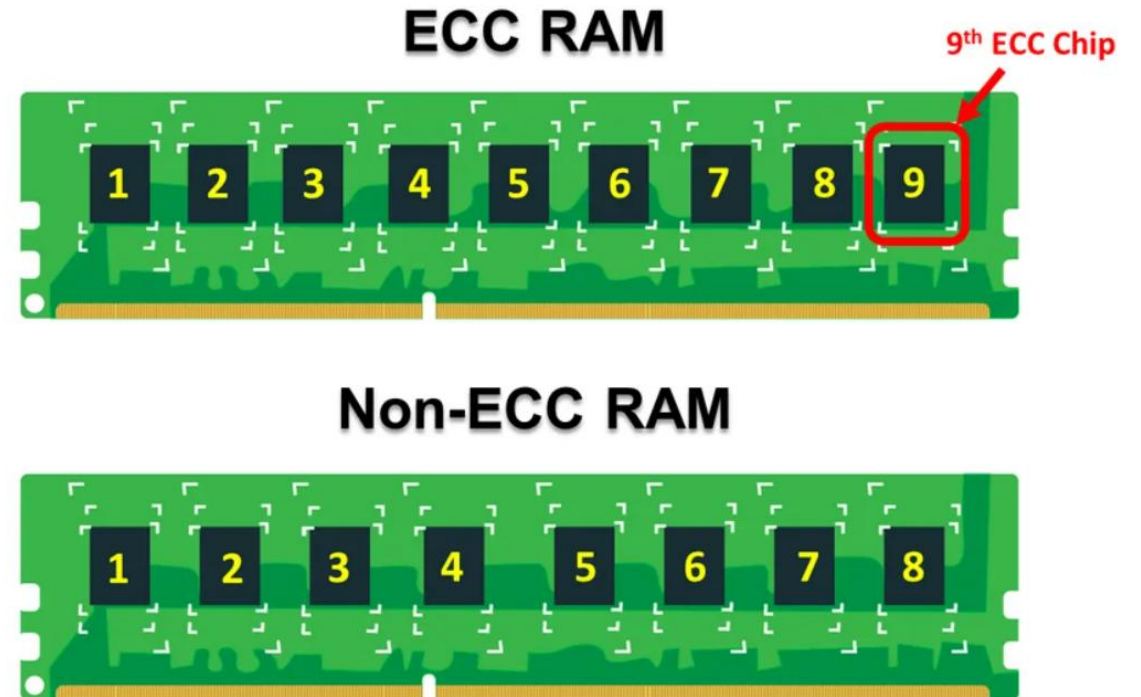
Failure Case with an additional soft-fault:

0	0	0	1	0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

- Extra bits used to store data to determine if there is an error.
- Can sometimes recover from a single bit-flip.
- Question of how many bits to use in ECC vs. memory, as impacts efficiency.

ECC Memory in Servers

- Not just embedded systems – “server-grade” memory often has ECC.
- Detects & can fix a certain number of bit flips.



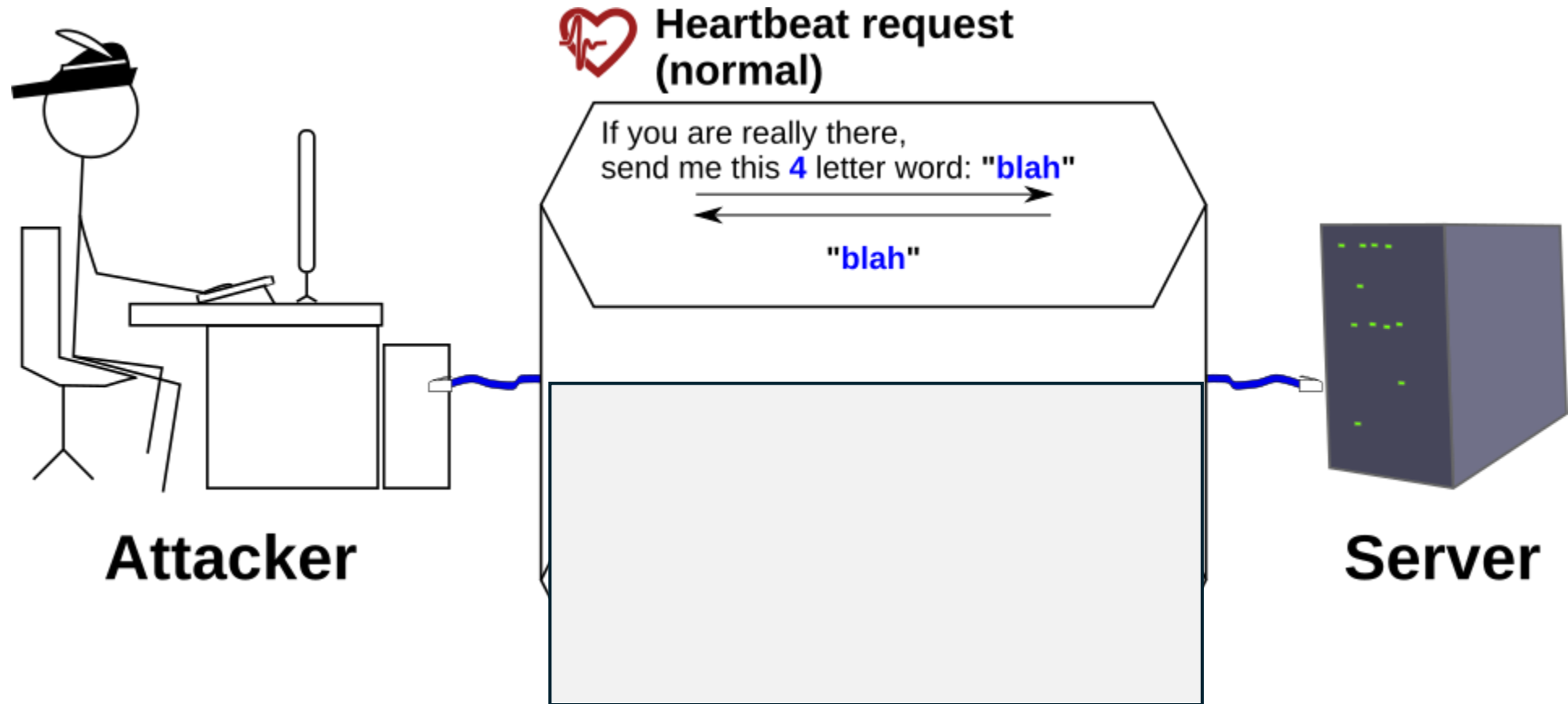
Microcontroller ECC Memory

- Certain memory types (FLASH memory) often contain ECC as they degrade with use. This will be (normally) hidden from you.
- ECC memory in microcontroller RAM is relatively rare for general use.
- Microcontrollers designed for safety-critical use (automotive) *very likely* to include these features.

Invalid Memory Access Problems

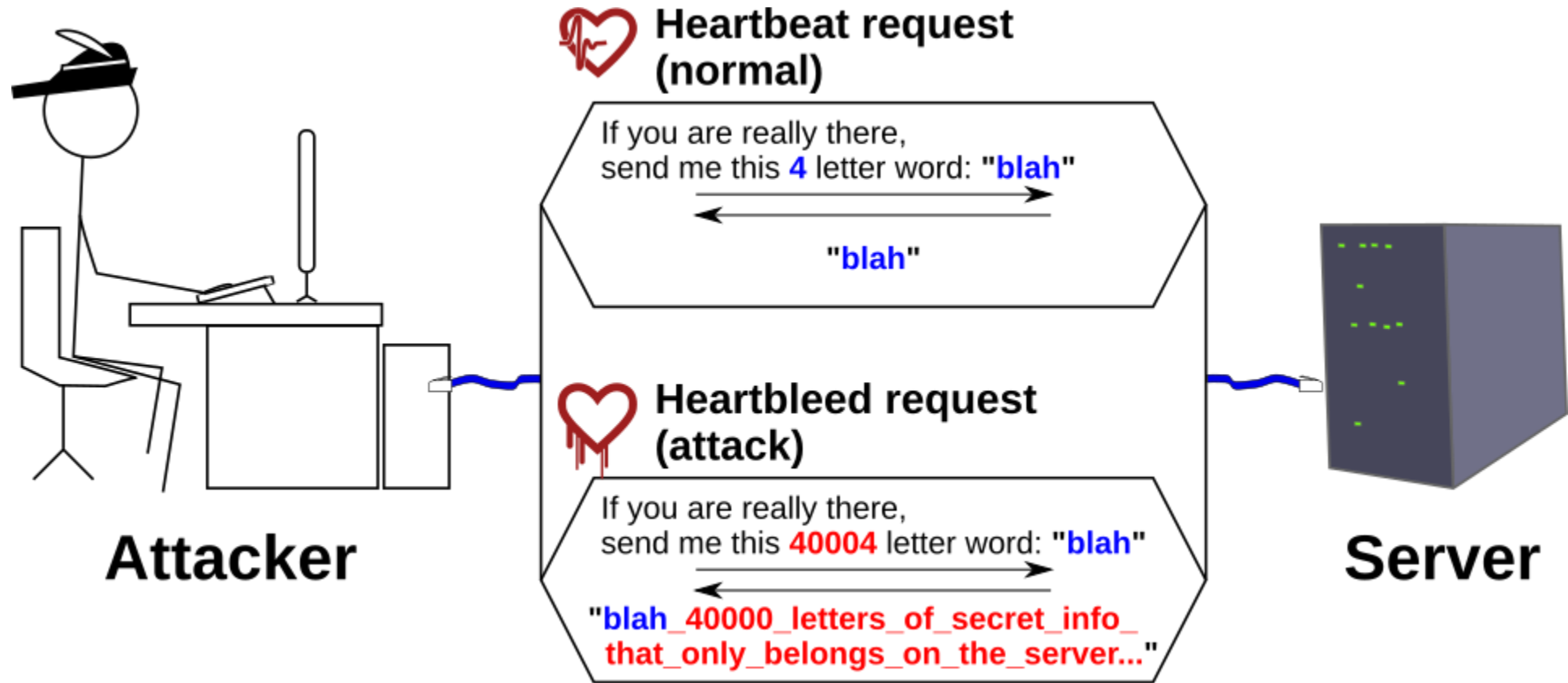
- Memory corruption is one problem, but what about:
 - Microcontroller tries to write somewhere invalid?
 - Microcontroller tries to execute from an area that isn't actually code?

e.g., HeartBleed



https://commons.wikimedia.org/wiki/File:Heartbleed_bug_explained.svg

e.g., HeartBleed

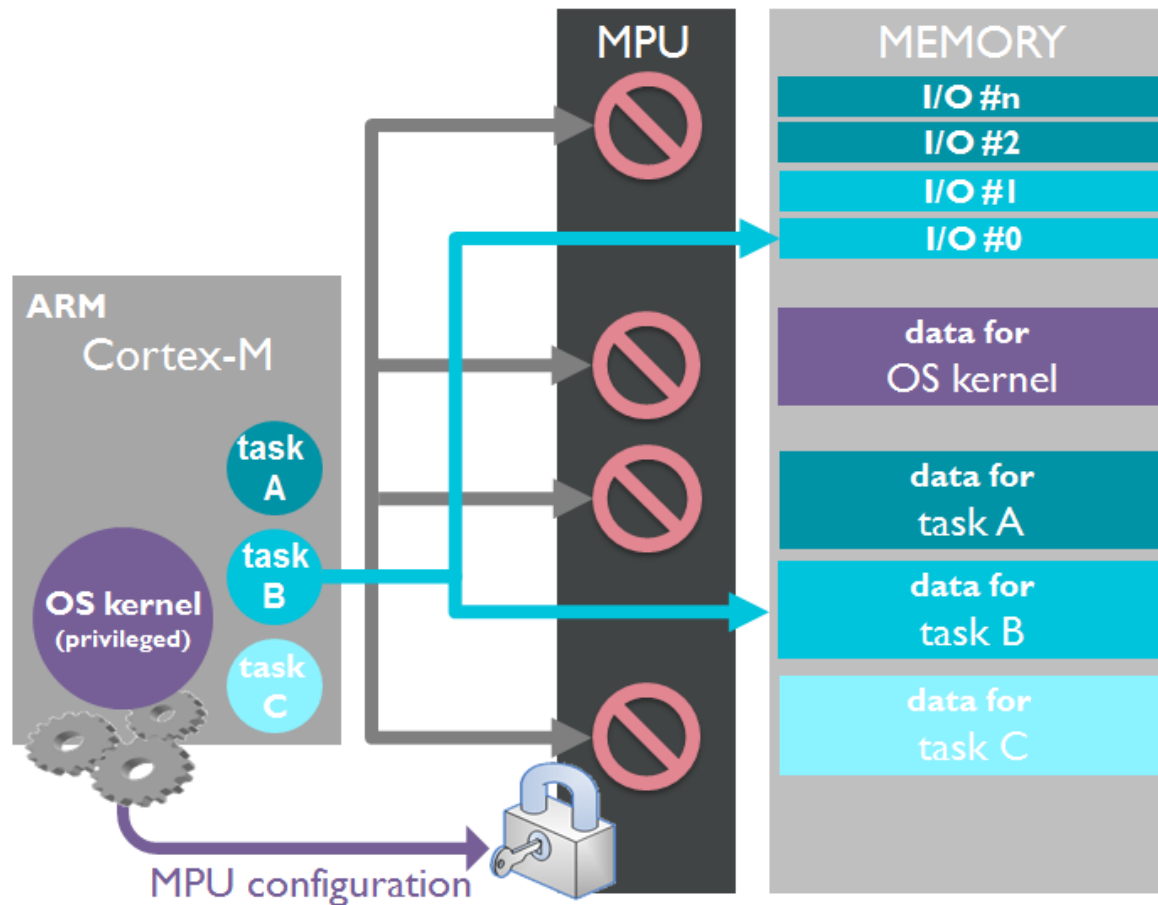


https://commons.wikimedia.org/wiki/File:Heartbleed_bug_explained.svg

Memory Overwriting & Reading

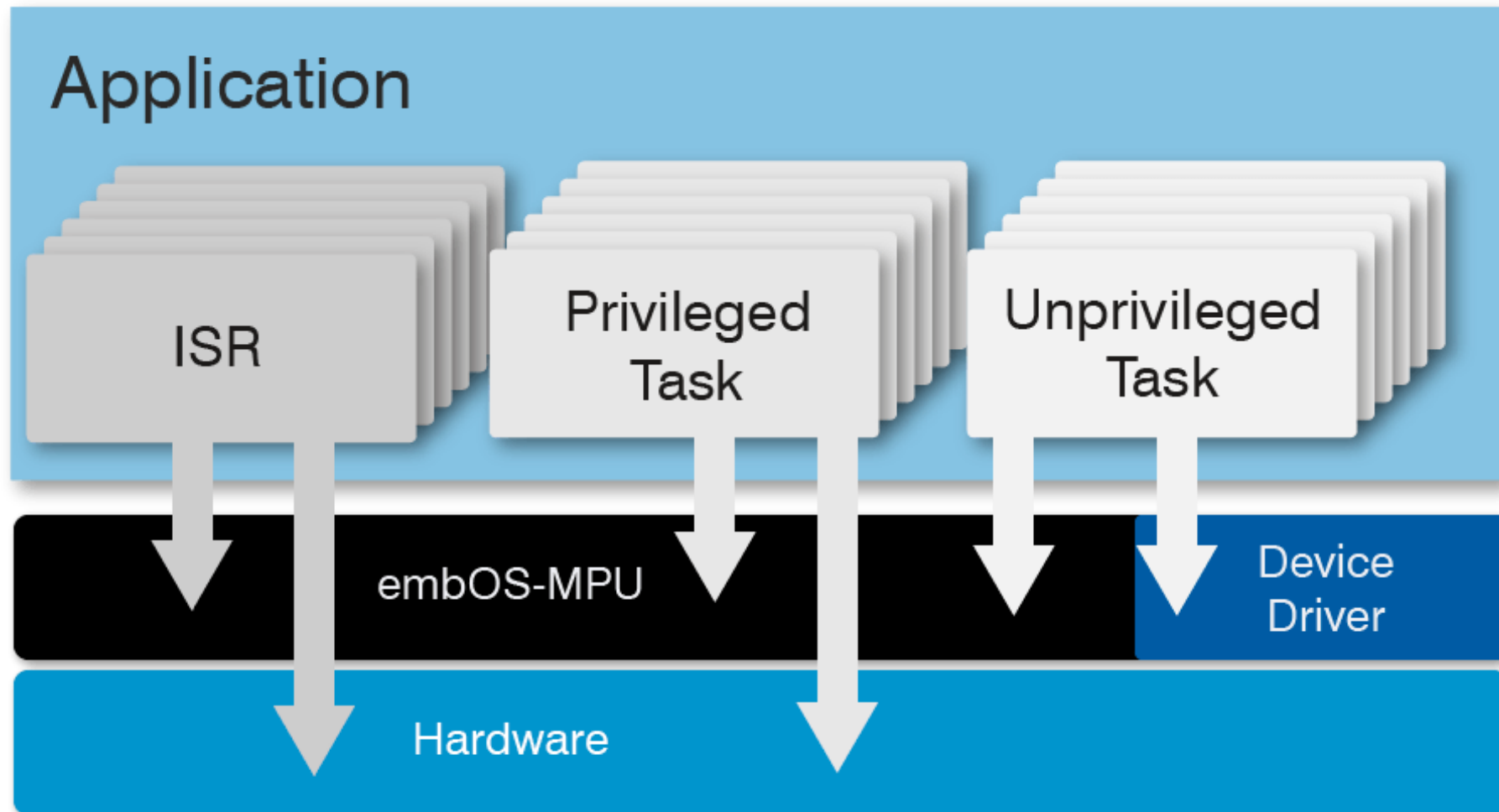
- Lots of examples of this in practice – but in general an attacker tries to write to or read from memory they should not be allowed to access.
- Often an attacker wants to execute some buffer they have written a program to.

Hardware Memory Protection



- Ideally each *task* would only have access to certain data.
- The capabilities of the MPU depend on the device.
- Minimum supports at least two modes

Privileged & Unprivileged Modes



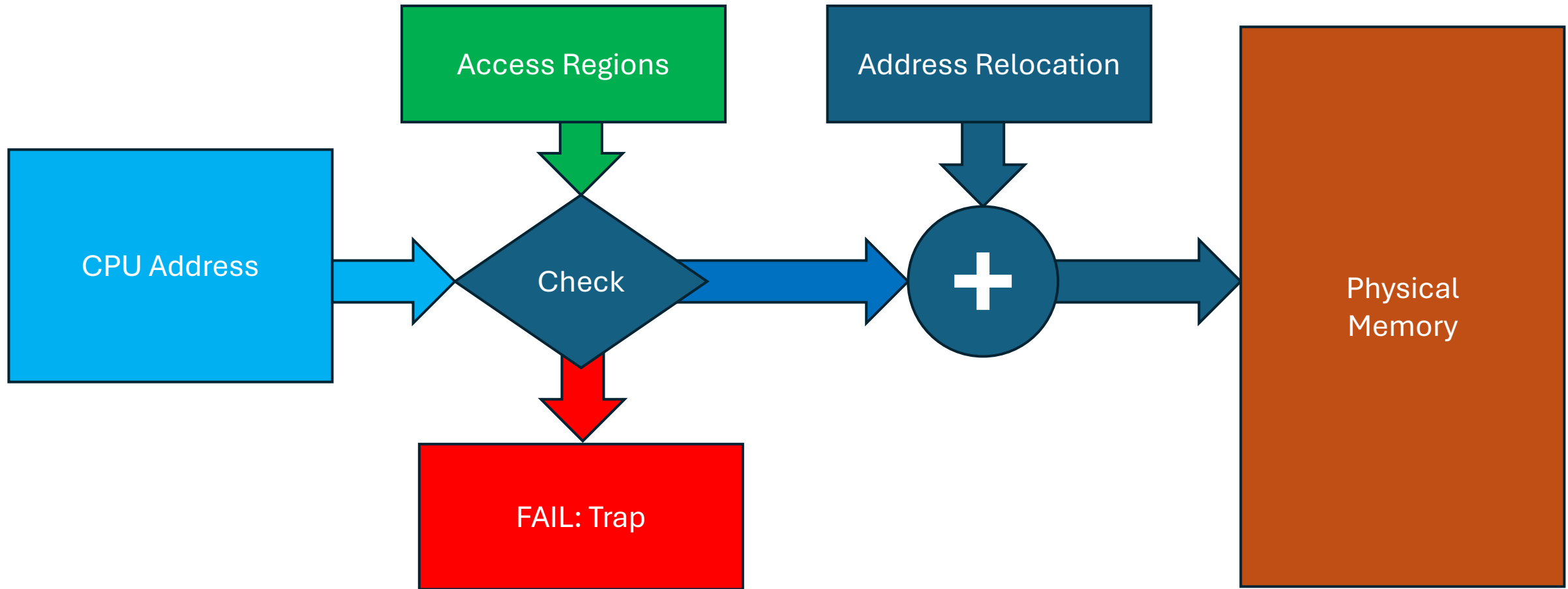
Read / Write / Execute Permissions

- Similar to on e.g., Linux:
 - Files can have permission for “read”, “write” and “execute”
 - Permissions can vary for owner or others (e.g., privileged vs. unprivileged)
- MPUs enforce this over address spaces.
- You need to carefully manage your address spaces to help with this.

Note: Harvard Architecture

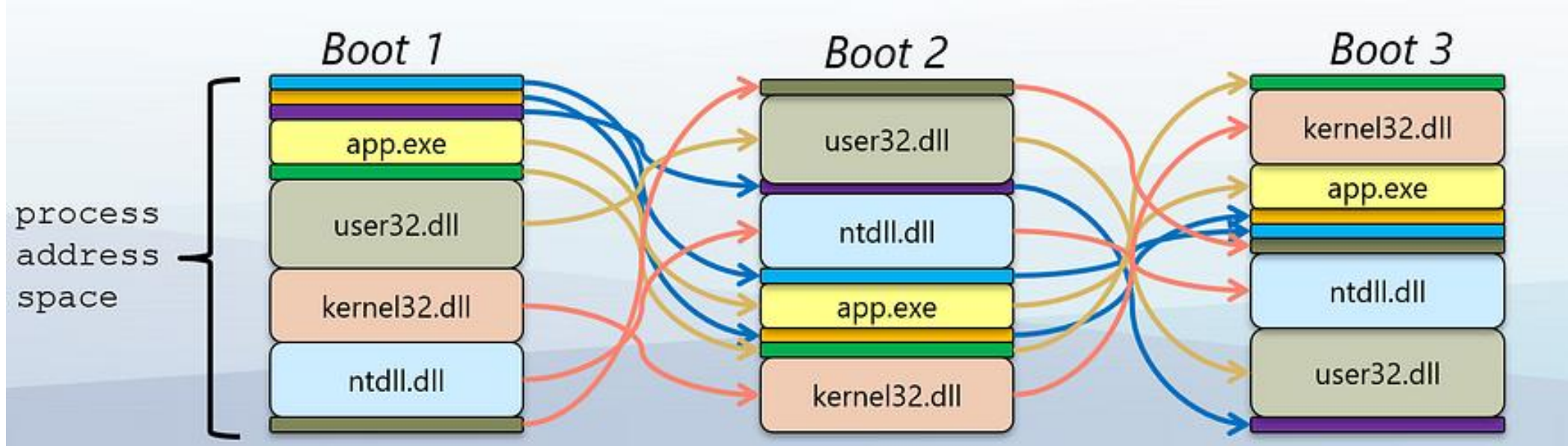
- Harvard architectures naturally enforce non-execute on data
 - Impossible in all cases to execute from the data region!
- Arm by default *allow* execution from data region (even if you never use it).

Memory Access Logic



Address Space Layout Randomization (ASLR)

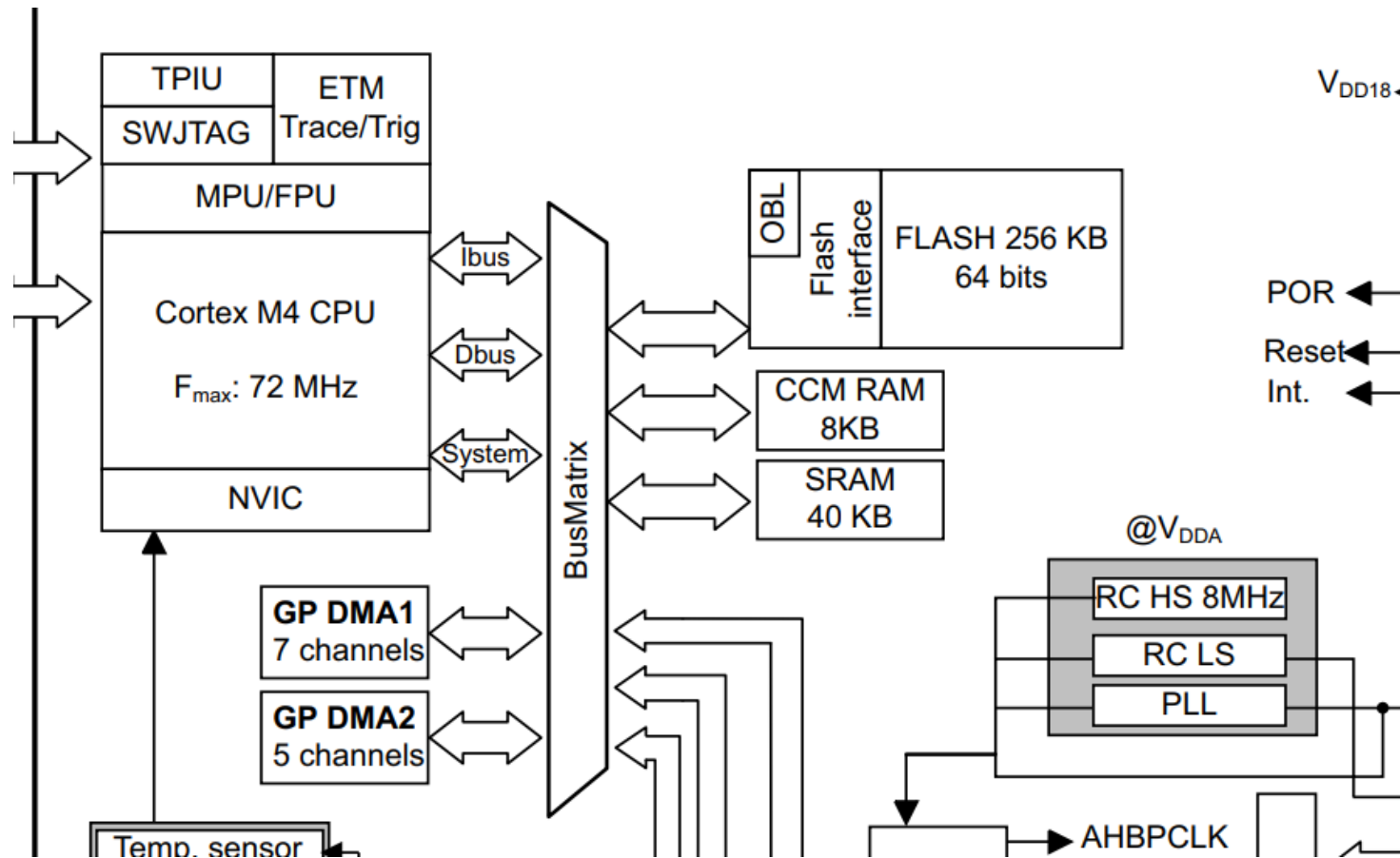
- Now common on Oses (not microcontrollers) – randomly changes where processes are in memory
- Makes it difficult (impossible) for attacker to exploit normal orders



MPU & MMU

- MPU (Memory Protection Unit)
 - Normally part of microcontrollers
 - Fixed number of regions we can configure
- MMU (Memory Management Unit)
 - Normally part of “application-class” processors (e.g., Running Linux/Android/Oses)
 - Address virtualization support, many additional features

MPU Example



- Note MPU is part of the core
- DMA (another bus controller) **is not bound by MPU!**

STM32F3 MPU Setup

```
// -----  
// Region 1: SRAM RW, XN (no execute)  
// Base 0x20000000. Size: 64KB for demo; adjust if needed.  
// AP: priv RW/unpriv RW (0b011), XN=1  
// -----  
mpu_config_region(  
    1,  
    0x20000000u,  
    64u * 1024u,  
    (1u << MPU_RASR_XN_Pos) |  
    (0b011u << MPU_RASR_AP_Pos)  
);
```

- E.g., disable execution from RAM

STM32 MPU Introduction

https://www.st.com/resource/en/application_note/an4838-introduction-to-memory-protection-unit-management-on-stm32-mcus-stmicroelectronics.pdf

https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

STM32 MPU Demo

<See Segger>

Memory Safe Languages

- C is not memory safe as you can do this:

```
char buffer[8];  
buffer[8] = 'x';
```

- You may get a compiler warning in that case, but if passed as a pointer there would be no warning.
- This is the exact bug type that caused the HeartBleed attack earlier.

More Invalid C Demos

```
char * return_invalid(void){  
    char buffer[128];  
    return buffer;  
}
```

```
void print_invalid(char * s){  
    putc(*(s + 100));  
}
```

Memory Safe Languages

- Hardware cannot prevent many of these problems – MPU would see them as valid.
- Memory-safe language guarantees:
 - No buffer overflow
 - No use-after-free
 - No dangling pointers
 - No invalid pointer arithmetic

Memory Unsafe Language Hints

- Generally memory-*unsafe* languages allow you to:
 - Directly manipulate pointers
 - Cast any value to any other value
 - Perform manual memory management

Memory Safe Languages

- E.g., a few languages & how they achieve that goal:

Language	Approach
Java	Runtime checks
Python	Runtime checks
Go	Garbage collection
Rust	Compile-time safety
Swift	Strong type system

Runtime Memory Safety

Enforced at runtime – not detected until you try and execute the code:

- Bounds checking
- Null checks
- Garbage collection
- Type Safety

```
>>> a = [1,2,3]
>>> a[10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Compiler-Time Memory Safety

- Rust prevents errors before execution
- Perfect for embedded systems where can be difficult to see error lots.
- Concepts of this mean tracking:
 - Ownership
 - Borrowing
 - Lifetimes

Rust Ownership Model

- Every value was one owner.
- Ownership can move.
- References must following borrowing rules.
- Lifetimes are checked and must be valid.

See next page for example

e.g., Rust compiling an error

```
1 fn main() {  
2     let x = String::from("hello");  
3     let y = x;  
4     println!("{}", x); // compile error  
5 }
```

Execution
Errors
Exited with status 101

Standard Error

Compiling playground v0.0.1 (/playground)
error[E0382]: borrow of moved value: `x`
--> src/main.rs:4:20

```
2 |     let x = String::from("hello");  
  |         - move occurs because `x` has type `String`, which does not implement the `Copy` trait  
3 |     let y = x;  
  |         - value moved here  
4 |     println!("{}", x); // compile error  
  |                   ^ value borrowed here after move
```

= note: this error originates in the macro ``$crate::format_args_nl`` which comes from the standard library
help: consider cloning the value if the performance cost is acceptable

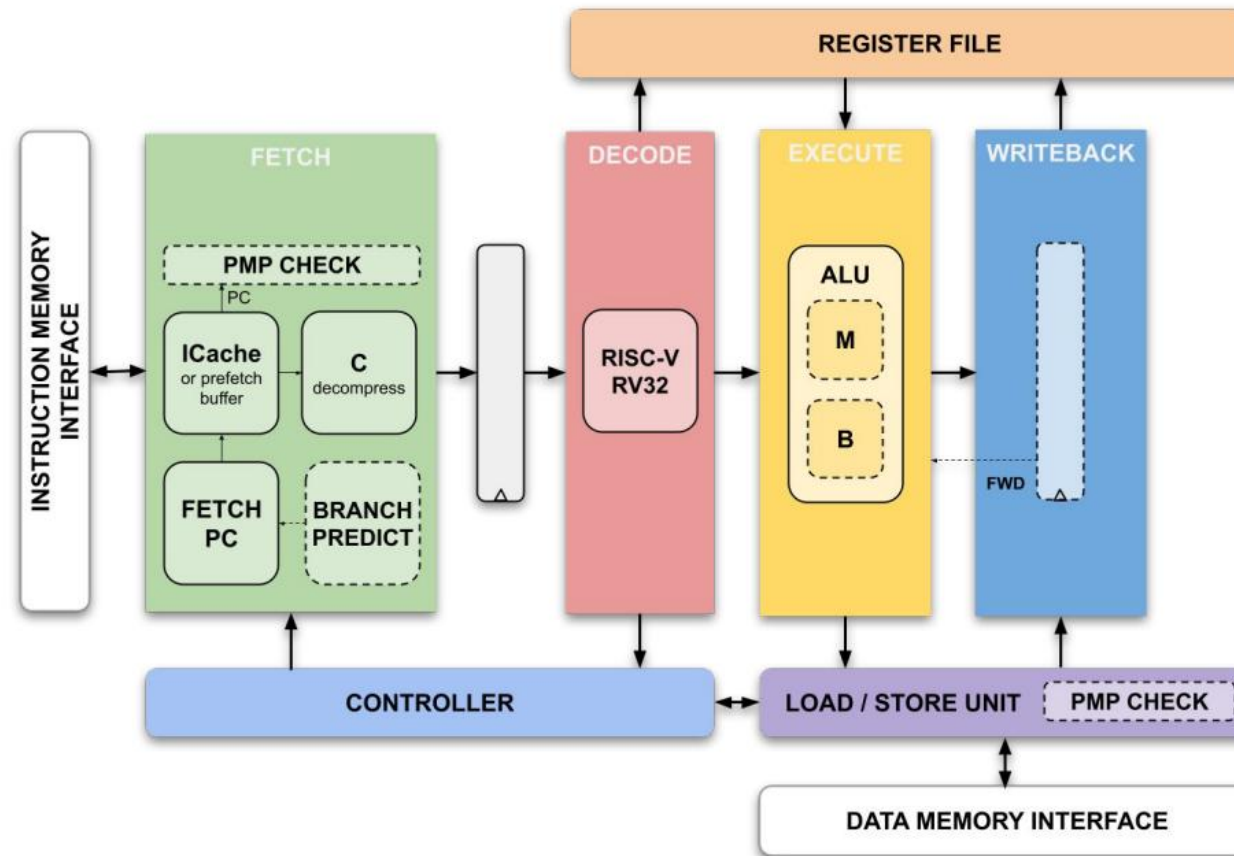
```
3 |     let y = x.clone();  
  |             ++++++
```

<https://play.rust-lang.org/?version=stable&mode=debug&edition=2024>

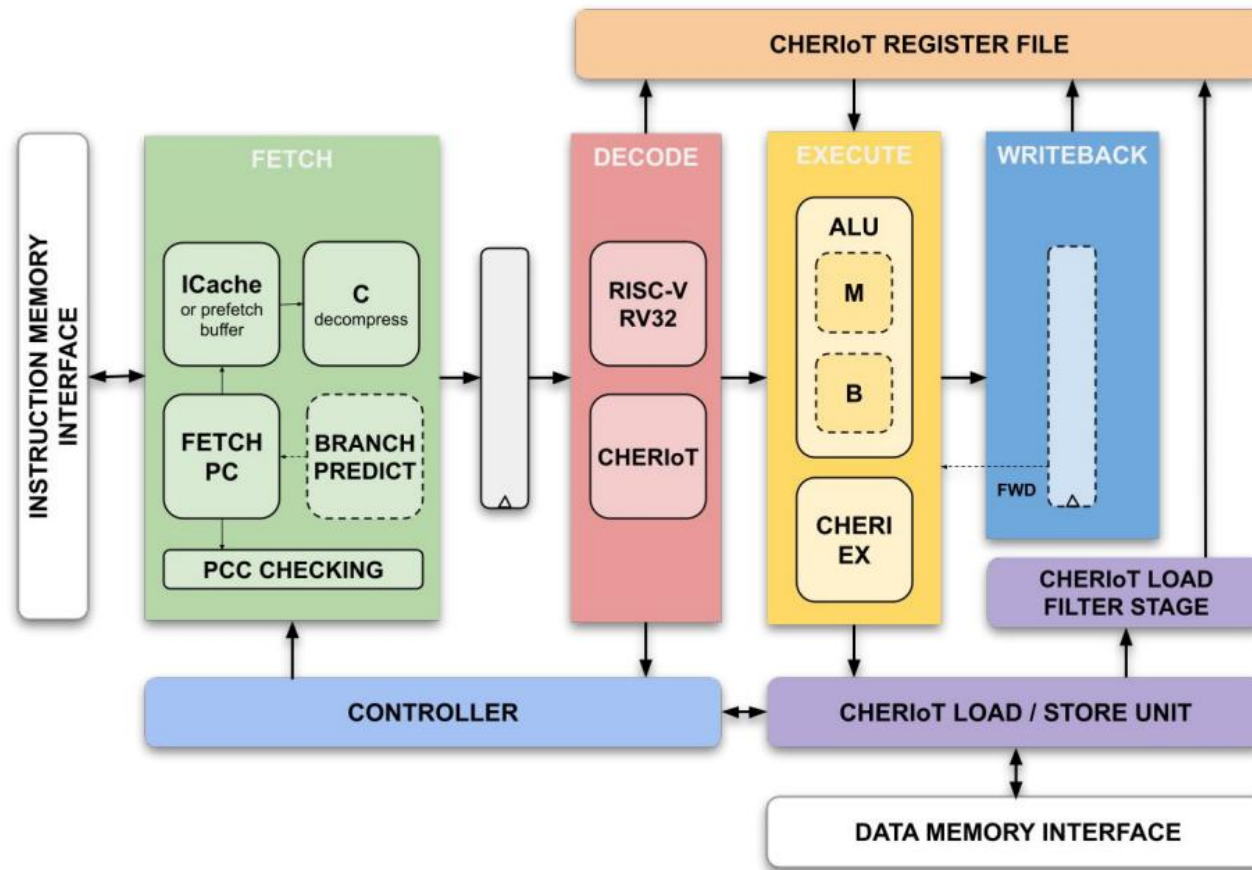
Memory Safe CPU Architectures

- New designs of processors try to enforce some of these concepts in hardware.
- Capability Hardware Enhanced RISC Instructions (CHERI) and CHERIoT are two variants of this idea (higher & lower performance)
- In CHERI pointers become capabilities:
 - Capabilities in hardware track address, bounds, permissions, and validity

CPU Core: Initial



CPU Core: Initial



e.g., CHERIoT Code (Programmer View)

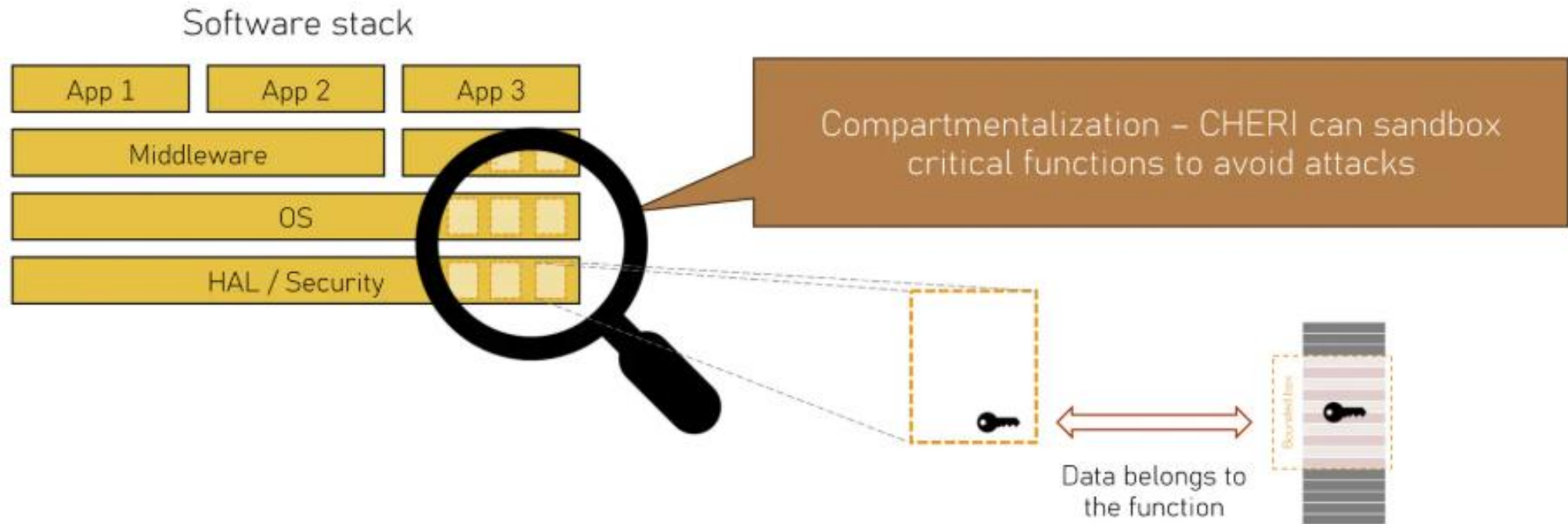
```
allocation = static_cast<char *>(malloc(length));
Debug::Assert(allocation != NULL,
               "Allocation failed in HeapLinearOverflow");

Debug::log("Trigger heap linear overflow");
allocation[length] = '\x41';

Debug::Assert(false, "Code after overflow should be unreachable");
```

https://github.com/CHERIoT-Platform/cheriot-rtos/blob/main/examples/08.memory_safety/memory_safety_inner.cc#L69

Other Features: Capabilities and Compartments



Hardware Enforced Memory Safety

- Unlike with programming language, the code will not run.
- Trade-off as may want to determine this at compile-time when that could be known.
 - Analysis tools can also determine this, leading to “best of both worlds”
- CHERI IoT is new – first physical devices demonstrated this year, but active development
- May be something you run across later in your career!

Memory Safety Summary

- Memory safety normally refers to preventing “out of bounds” access.
- Often presents a security risk, but also a safety risk if e.g., crashing program causes a physical device to misbehave.
- Memory Protection Unit (MPU) is most commonly used device in smaller microcontrollers to enforce region level access.
- Memory-safe languages (such as Rust) and architectures (such as CHERIoT) gaining wider acceptance in embedded.